Applying Mixture of Recursions (MoR) to Vision Transformers (ViT)

- **Introduction**:

- Vision Transformers (ViTs) have demonstrated state-of-the-art performance in image classification, detection, and related vision tasks.
- Their strength comes from global self-attention, which captures long-range dependencies effectively.
- However, conventional ViTs adopt a uniform depth strategy where all tokens undergo the same sequence of layers, regardless of complexity.
- This design leads to computational redundancy and limits deployment efficiency.

- **Motivation**:

- In real images different tokens carry different semantic importance (eg: object regions vs background).
- Uniform processing wastes computation on trivial or redundant tokens.
- Resource constrained applications (mobiles… etc) demand adaptive models that allocate compute efficiently.
- The Mixture of Recursions (MoR) framework addresses this by enabling token-wise dynamic recursion depth, allowing important tokens to be processed deeply while others exit early.
- MoR-ViT integrates this principle into ViTs, balancing efficiency and accuracy.

Literature review: Attention is All You Need

The Original Transformer Architecture: Introduced for NLP tasks, it relies on a self-attention mechanism to weigh the importance of different tokens in a sequence.

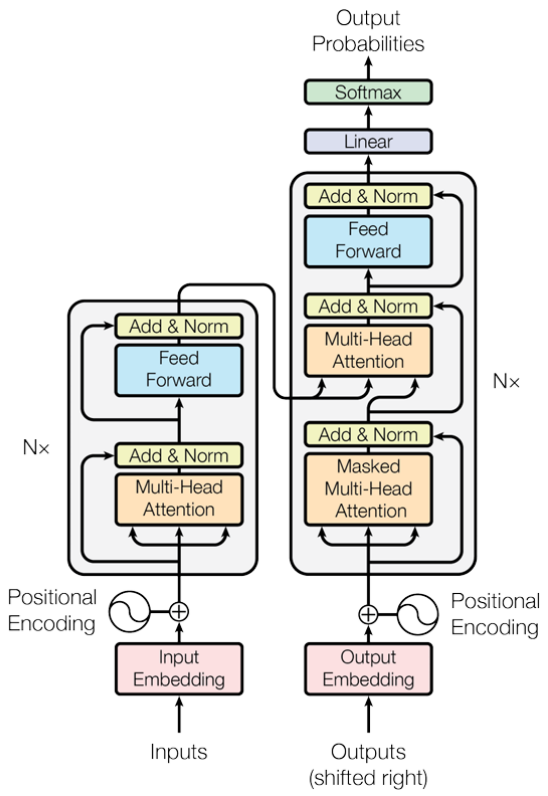
- Self-Attention Mechanism: The core of the Transformer's success lies in its attention formula:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

- Where Q(Query), K(Key), and V(Value) are learned linear projections.

- **Gaps in the existing solutions/work**:

- **Parameter-Heavy:** Each layer has its own unique set of weights, leading to very large models.
- **Uniform Depth:** Every token passes through the same fixed number of layers.
- **No Adaptivity:** Lacks a mechanism to adjust computation based on input complexity



Literature review: Relaxed Recursive Transformers(RRT)

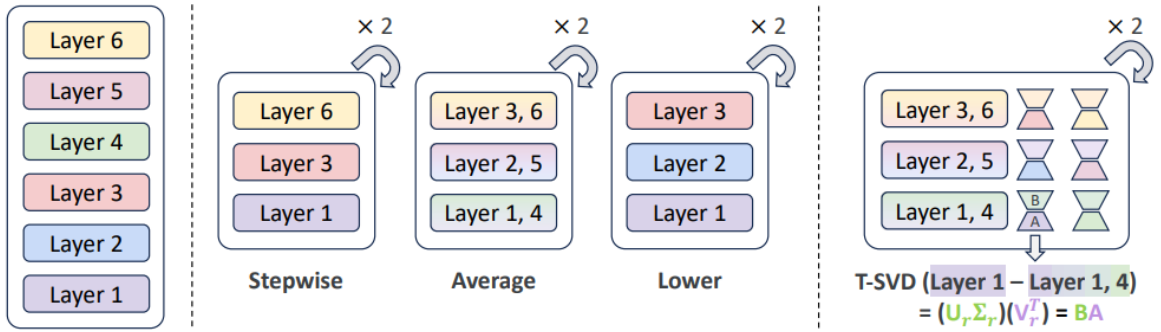
- Instead of stacking Lunique transformer layers, recursive models reuse the same block multiple times to increase depth with fewer parameters.
- Update Equation (Recursive Transformer): where $\Delta\Phi^{(r)} = A^{(r)}B^{(r)}$is a low-rank update (LoRA).

$$\mathbf{h}_t^{\ell} = f(\mathbf{h}_t^{\ell-1};\ \Phi'_{((\ell-1)\bmod L/B)+1}, \Delta\Phi'_{\ell}),\ \ \ell \in [1, L]$$

- Introduces layer-wise Low-Rank Adaptation (LoRA) to relax strict parameter sharing, giving each recursion step a small adaptation.
- where $r \in \{1, \ldots, N_r\}$is recursion step, and $\Phi'$are shared parameters.

- **Gaps in the existing work**:

- Applies recursion depth uniformly to all tokens.
- KV cache overhead grows with depth.
- No adaptive per-token computation.



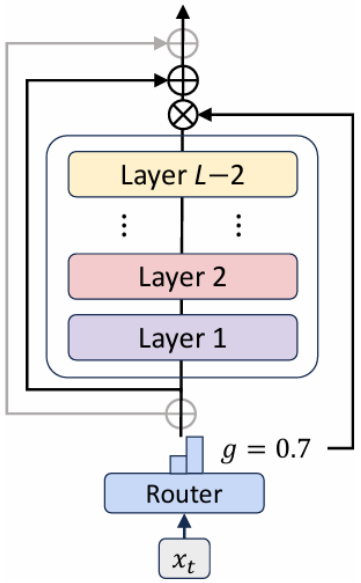
Literature review :MoR (Mixture-of-Recursions)

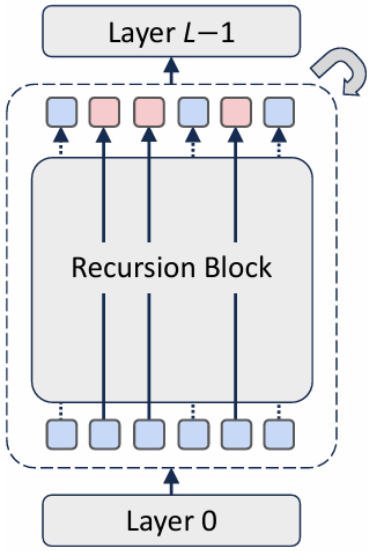
Unifies two efficiency paradigms:

- Parameter sharing (recursive blocks).
- Adaptive computation (token-level routing).
- Uses routers to assign token-specific recursion depths.
- Equation:

$$H_t^{r+1} = g_t^r f(H_t^r; \Phi') + \text{(residual)}$$

- Adds KV caching for memory efficiency.





Two Routing Strategies:

- Expert-choice :Each recursion step picks its top-k tokens (step-wise decision).
- Token-choice : Each token commits to a recursion depth at the start (token-wise decision).

Two KV Caching Strategies:

- Recursion-wise caching :Cache only active tokens at each recursion step.
- Recursive sharing : Cache all tokens once at the first step and reuse across recursions.

Expert-choice Routing:

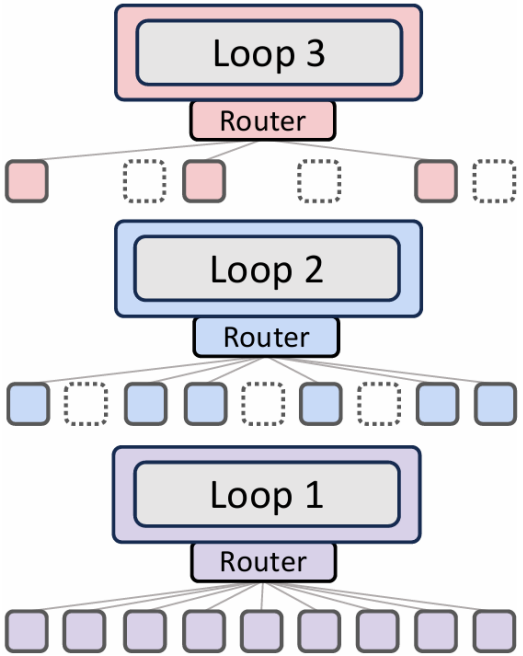
- Each recursion step = one expert.
- Router score:

$$g_t^r = \mathcal{G}(\theta_r^\top \mathcal{H}_t^r)$$

- $\mathcal{G}$ represents an activation function.
- Top-k tokens are selected for next step.
- Residual:

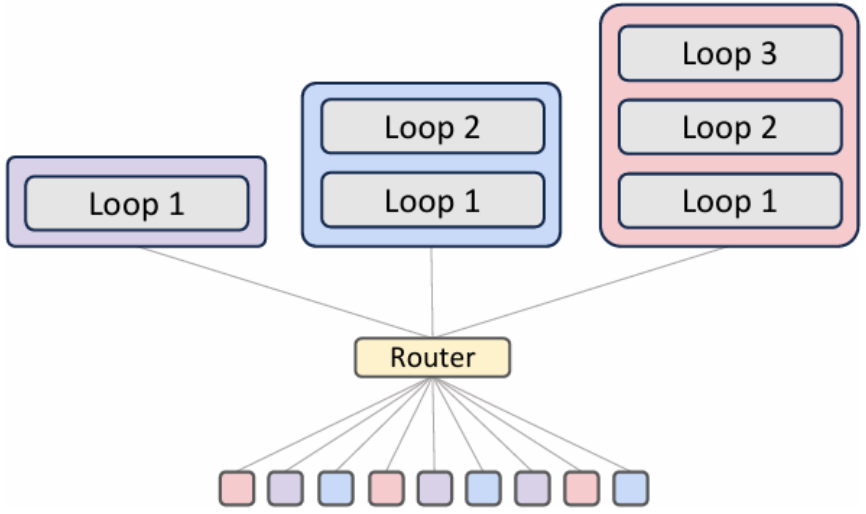
$$\mathcal{H}_t^{r+1} = \begin{cases} g_t^r f(\mathcal{H}_t^r, \Phi') + \mathcal{H}_t^r, & \text{if } g_t^r > P_\beta(G^r) \\ \mathcal{H}_t^r, & \text{otherwise} \end{cases}$$

- where $P_\beta(G^r)$ is the-percentile threshold over all scores at recursion step r



Token-choice Routing:

- Each token commits once to depth i.
- Decision: $i = \arg\max_j g_t^j$
- Residual: $$\mathcal{H}_t^{r+1} = \begin{cases} g_t^r f(\mathcal{H}_t^r, \Phi') + \mathcal{H}_t^1, & \text{if } r = i \\ g_t^r f(\mathcal{H}_t^r, \Phi'), & \text{otherwise} \end{cases}$$



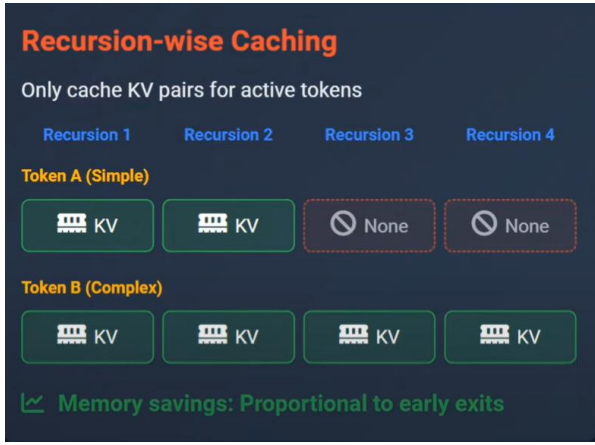
- Here $\mathcal{G}$ is a non-linear function (softmax or sigmoid)
- $g_t = \mathcal{G}(\theta_r^\top \mathcal{H}_t^1)$ where gj t denotes the routing score for expert j ∈ {1,…. Nr}.

KV Caching:

Recursion-wise Caching:

At each recursion step, only active tokens cache their key-value(KV) pairs.

- Tokens that exit early are not cached further.
- Attention is restricted to those cached tokens at that depth.
- Memory/IO cost ≈ $\frac{N_r+1}{2N_r}$.
- FLOPs ≈ $(kN_{ctx})^2$.
- $Where$ Nctx = context length (no.of input tokens)



Recursive KV Sharing:

- Cache all tokens once at the first recursion step.
- Reuse this cache across all deeper recursions.
- Even if tokens exit early their original KVs remain accessible.

Memory ≈ $1/N_r$

- FLOPs ≈ $k\,.\,N_{ctx}$

Proposed Work :MoR-ViT Architecture - Integration

- Goal: To apply the adaptive computation benefits of MoR to Vision Transformers.
- MoR-ViT Integration:
  - The process begins like a standard ViT.
  - Patch Embedding: Given an input image $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$, the image is first divided into N non-overlapping patches of size P×P, where $N = HW/P^2$. Each patch is flattened and projected into a D-dimensional token embedding:

$$\mathbf{x}_p = \mathrm{Flatten}(\mathbf{X}_p)\,\mathbf{W}_e + \mathbf{b}_e, \quad p = 1, \ldots, N$$

  - Input Sequence Creation: A learnable class token $\mathbf{x}_{\mathrm{cls}} \in \mathbb{R}^D$ is prepended, and positional embeddings Epos are added to create the final input sequence

$$\mathbf{Z}_0 = [\mathbf{x}_{\mathrm{cls}}; \mathbf{x}_1; \ldots; \mathbf{x}_N] + \mathbf{E}_{\mathrm{pos}}$$

  - Adaptive Processing: This sequence Z0 is then fed into a single MoR block, which adaptively processes each token. The final [CLS] token is used for classification..

Proposed Work :MoR-ViT Architecture- Routing Mathematics

- **Routing Mechanism:** MoR-ViT uses Expert-choice routing to dynamically select which tokens continue recursion.
- **Gating Score:** At each recursion step l, a lightweight router computes a gating score gt(l) for each token's hidden state ht(l). Where σ is the sigmoid activation.

$$g_t^{(l)} = \sigma\left(\mathbf{w}^\top h_t^{(l)} + b\right)$$

- **Dynamic Masking:** A binary mask mt(l) is created by thresholding the gating scores at an adaptive β-th percentile.

$$m_t^{(l)} = \begin{cases} 0, & \text{if } g_t^{(l)} < \text{Percentile}_\beta(\{g_{t'}^{(l)}\}_{t'=1}^N) \\ 1, & \text{otherwise} \end{cases}$$

- This allows tokens corresponding to simple image regions to exit early, while complex regions receive deeper processing.

Proposed Work :MoR-ViT Architecture - Training & Benefits

- **Final Loss:**

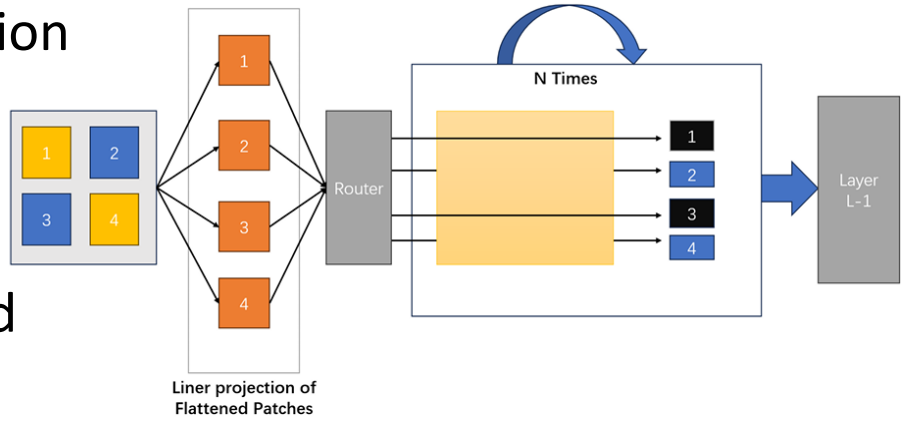
▪The model is trained end-to-end with a combined loss function that balances task performance and routing stability.

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda \mathcal{L}_{\text{routing}}$$

▪L task is the primary classification loss (Cross-Entropy).

▪L routing is a regularization term to encourage balanced and stable routing decisions.

▪λ is a hyperparameter to control the trade-off.



- **Key Benefits & Results Summary:**

- Less Compute & Fewer Parameters: Achieves up to 68% parameter reduction and 2.0x inference acceleration.
- Adaptive Depth: The first ViT to enable token-level dynamic recursion, allocating resources more efficiently.
- State-of-the-Art Performance: Outperforms leading efficient ViT baselines, validating dynamic recursion as a powerful strategy for scalable vision models.

References:

- *Vaswani, A., et al. Attention is All You Need. NeurIPS, 2017. arXiv:1706.03762*
- *Xu, H., et al.Relaxed Recursive Transformers: Effective Parameter Sharing with Layer-wise LoRA. arXiv:2410.20672, 2024. arXiv:2410.20672*
- *Yao, Z., et al.Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. arXiv:2406.15524, 2024.*
- *Yao, Z., et al. MoR-ViT: Efficient Vision Transformers via Mixture of Recursions. arXiv:2507.10524, 2025. arXiv:2507.10524*